

The image features the text "DL for ENG" in a bold, 3D, gold-colored font. The letters are floating on a deep blue ocean with visible ripples and small waves. In the background, there are rolling blue hills or mountains under a sky filled with soft, grey clouds. The overall scene has a serene, digital aesthetic.

DL for ENG

LECTURE 5.1

Convolutional and Graph Networks

One learned local rule, applied everywhere: on a regular grid it is a convolution, on an arbitrary graph it is message passing.

Remus Teodorescu

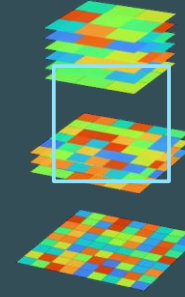
With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

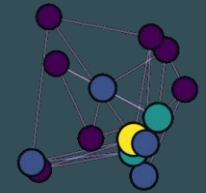
Deep Learning for Engineering · 2026

convolution · message passing · permutation equivariance · the six-bus network

THE SAME RULE, TWO DOMAINS



a grid — every pixel has the same neighbours



a graph — whoever the edges connect

Left: nine neighbouring pixels, and one kernel applied to every such neighbourhood. Right: six buses, and one update rule applied at every node. Different pictures, one idea.

Learning Objectives

SECTIONS, IN ORDER

- 1 **Local rules** — why a dense layer on an image is wrong, and the rule that replaces it
- 2 **Convolution** — what a convolutional network is; kernel, parameters, pooling, equivariance
- 3 **Residual connections** — how depth is made to train
- 4 **Graphs** — what a graph network is; matrices, message passing, permutation equivariance
- 5 **Choosing the architecture** — when a graph network earns its place, and when not
- 6 **Challenges** — what goes wrong first, and what to check

WHAT WE SKIP, AND WHERE IT IS TAUGHT

transfer learning	reusing early layers — L6.2
camera network	a CNN on the JetRacer — L11.1
physics on a graph	the six buses with a residual — L12.1
sequences	RNN, LSTM, attention, transformers — L5.2

The six-bus network of this lecture is the one you build on in Ex_05 notebook 03 and again in Ex_12.1.

Recap

THE TEN QUESTIONS OF L4.2 — DEEP NEURAL NETWORKS

- 1** What is a deep network, written as a composition, and what does one layer compute in matrix form? *UDL ch. 4.1 – 4.4*
- 2** Why does doubling the width cost four times the parameters while doubling the depth costs two? *UDL ch. 4.4; Ex_04 notebook 04*
- 3** Why do the linear regions multiply rather than add when layers are composed, and what does that say for depth? *UDL ch. 4.5*
- 4** What is the vanishing gradient, and which three changes made deep networks trainable? *UDL ch. 7.5, 11*
- 5** What is model capacity, and how do training and held-out error behave as it grows? *UDL ch. 8.1*
- 6** A model passes through every training point. Why is that not good news, and how would you detect it? *UDL ch. 8.1; Ex_04 notebook 05*
- 7** What are the jobs of the training, validation and test sets, and why is the test set used once? *UDL ch. 8.1*
- 8** What are the three terms in the bias–variance decomposition, and what does an error below the noise floor suggest? *UDL ch. 8.2*
- 9** What does double descent add to the classical U-curve, and when does it matter in practice? *UDL ch. 8.4; Belkin et al. (2019)*
- 10** Given an overfitted model, why regularise before shrinking it, and which two methods would you try first? *UDL ch. 9.1; Ex_04 notebook 05*

Local Rules

A DENSE LAYER ON AN IMAGE, AND WHAT REPLACES IT

- **a 256×256 image** — 65,536 inputs; dense to 1,000 neurons, 65 million weights
- **the larger problem** — the layer does not know that neighbouring pixels are related
- **the rule** — a small function of a neighbourhood, learned once, applied everywhere
- **on a grid** — the neighbourhood is a window: convolution
- **on a graph** — the neighbourhood is whoever the edges connect: message passing
- **structure you know** — built into the architecture is free; learned from data costs parameters and data

ON A GRID — CONVOLUTION

computes	a weighted sum over a $k \times k$ window
parameters	$F \cdot C \cdot k^2 + F$, whatever the image size
assumes	what matters is local and the same everywhere

ON A GRAPH — MESSAGE PASSING

computes	the same sum, over the listed neighbours
parameters	$C \cdot F + F$, whatever the number of nodes
assumes	no order among nodes, only connections

Convolutional Network

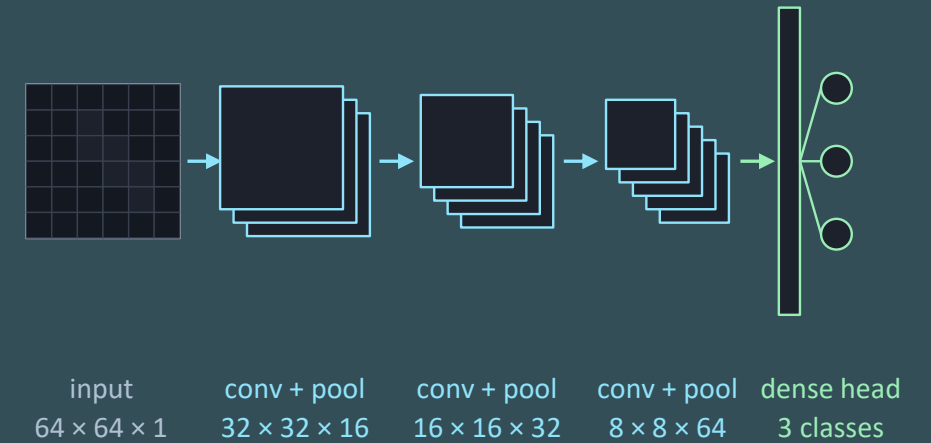
KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a feed-forward network whose layers are convolutions
- **function** — image in, features or label out, shift-equivariant
- **architecture** — convolution and pooling blocks, then a dense head
- **applications** — classification, segmentation, detection, 1-D signals

AGAINST A DENSE NETWORK

	dense network	convolutional
connections	every input to every neuron	a $k \times k$ window only
weights	one per connection	one kernel, reused at every position
parameters	grow with the input size	$F \cdot C \cdot k^2 + F$, any image size
a shifted input	an unrelated output	the same shift of the output
input size	fixed when designed	any, with global pooling

THE ARCHITECTURE, NOTEBOOK 01'S SIZES



Resolution falls, channels rise, and a dense head reads the last map. Redrawn after the typical CNN architecture of the Wikipedia article, with the sizes of notebook 01: under 100,000 parameters, a minute of training on a laptop.

Kernel

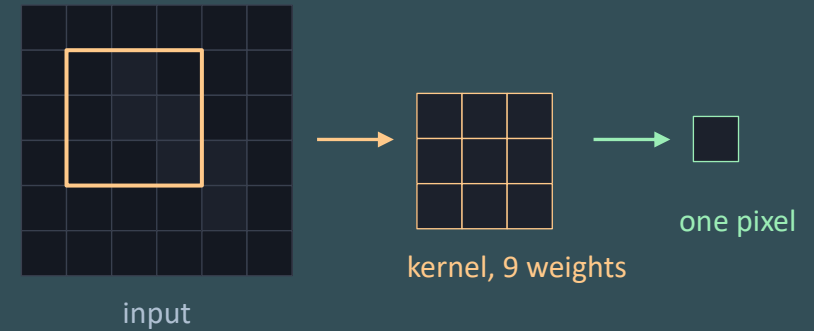
A WEIGHTED SUM OVER A SMALL WINDOW

- a 3×3 window — multiply, sum, add a bias, apply the activation
- move one pixel and repeat — with the same nine weights
- the nine weights are learned — before 2012 an engineer designed them by hand
- the output — an image of the same kind, one value per position

$$h_{ij} = a \left(\sum_m \sum_n k_{mn} x_{i+m, j+n} + b \right)$$

x the input image, k_{mn} the kernel, b one bias, a the activation, h_{ij} one output pixel

ONE WINDOW, ONE OUTPUT PIXEL



Slide the amber window over every position and the whole output image is built with the same nine numbers. That reuse is the saving, and it is why the detector works anywhere in the frame.

Convolution Parameters

STRIDE, PADDING, CHANNELS

- **stride** — how far the window moves; two halves the resolution
- **padding** — what happens at the edge; zeros keep the size
- **channels** — F output channels, each with its own $k \times k \times C$ kernel
- **the count** — $F \cdot C \cdot k^2 + F$, and it does not depend on the image size

$$P_{\text{dense}} = N_{\text{in}}N_{\text{out}} \quad P_{\text{conv}} = K^2C_{\text{in}}C_{\text{out}}$$

$N_{\text{in}}, N_{\text{out}}$ neurons in and out, K the kernel side, $C_{\text{in}}, C_{\text{out}}$ channels in and out

THE USUAL SETTINGS

kernel size	3×3 almost always
stride	1 to keep the size, 2 to halve it
padding	'same' keeps the size
channels	how many features per position

ONE LAYER, COUNTED

64 channels to 64 channels with 3×3 kernels: $64 \times 64 \times 9$, about 37,000 weights, independent of the image size. The same layer works on a thumbnail and on a gigapixel scan.

Pooling and Hierarchy

RESOLUTION DOWN, CHANNELS UP, FEATURES DEEPER

- **pooling** — each block becomes one number, usually its maximum
- **fewer positions** — less computation, a small insensitivity to shifts
- **resolution falls as channels rise** — position traded for meaning
- **the hierarchy** — edges, then textures, then parts, then objects
- **nobody designs it** — it emerges from the architecture and the loss
- **classification pools** — segmentation keeps position throughout

NOTEBOOK 01, IN OUTLINE

input	$64 \times 64 \times 1$
conv 3×3, pool	$32 \times 32 \times 16$
conv 3×3, pool	$16 \times 16 \times 32$
conv 3×3, pool	$8 \times 8 \times 64$
flatten, dense	4,096 to 3 classes

WHAT THE LAYERS LEARN

layer 1	oriented edges, blobs
layers 2 – 3	corners, junctions, texture
middle	parts — a wheel, a weld toe
late	whole objects, whole defects
output	the label

Equivariance

EQUIVARIANT AND INVARIANT

- **equivariant** — move the input, the output moves with it
- **invariant** — move the input, the answer does not change
- **convolution** — equivariant by construction
- **pooling, global averaging** — build invariance; position is discarded
- **which a task needs** — depends on whether its output should move

TALK · 2 MIN In the table on the right, which row describes a crack detector that must say where the crack is, and which a classifier that only says whether there is one?

WHAT EACH OPERATION GIVES

	property	used for
convolution	equivariant	every layer
pooling	towards invariance	classification
global average	invariant	one label per image
no pooling	equivariant	segmentation
message passing	equivariant to relabelling	graphs

The last row is the same property on a different symmetry: a graph network is equivariant to renumbering the nodes, and that is the argument for using one on a power network.

Residual Connections

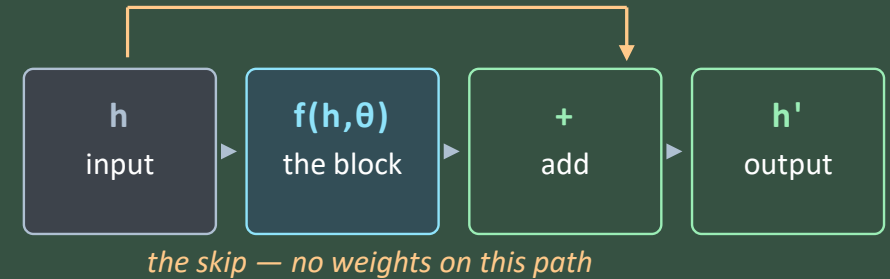
A PATH WITH NO WEIGHTS ON IT

- **vanishing gradient** — a long product of derivatives along the depth
- **residual connection** — add the block's input to its output
- **the gradient passes** — through the addition, past every block
- **each block learns a correction** — output zero when nothing to add
- **normalisation** — standardise per layer, rescale with learned scale

$$\mathbf{h}_{k+1} = \mathbf{h}_k + f(\mathbf{h}_k, \theta_k)$$

$\mathbf{h}_k$ the input to block k , f the block, θ_k its parameters, the other $\mathbf{h}_k$ is the skip

THE SKIP, DRAWN



Trainable depth went from about twenty layers to over a hundred within a year of this idea (He et al. 2016). It is in every deep architecture since, the transformer included.

NORMALISATION

batch norm	vision; model.eval() at inference
layer norm	transformers

Graph Neural Network

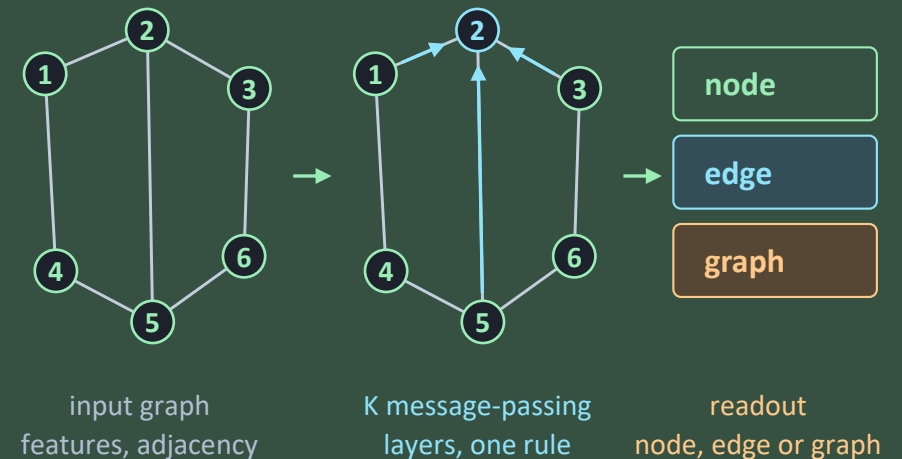
KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — layers that update every node from its neighbours
- **function** — graph in, an answer per node, edge or graph out
- **architecture** — K message-passing layers, then a readout
- **applications** — grids, meshes, pipe networks, molecules

AGAINST A DENSE NETWORK

	dense network	graph network
input	one vector, fixed length	a graph of any size, with features
connections	every input to every neuron	along the edges only
weights	one per connection	one rule, shared by every node
renumbered nodes	an unrelated output	the same output, renumbered
topology	fixed when trained	an argument; one model, many graphs

THE ARCHITECTURE, ON THE SIX BUSES



Node level: the voltage at every bus. Edge level: the flow on every line. Graph level: is the network stable. Applied to a graph never seen, it is inductive.

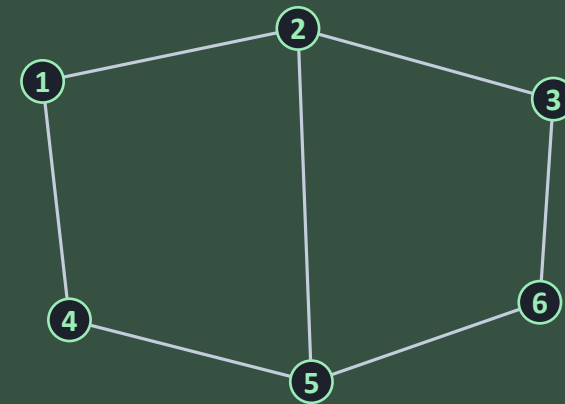
Irregular Neighbourhoods

THE SAME RULE, WITHOUT A GRID

- **convolution needs** — a regular neighbourhood with a left and a right
- **a power network has neither** — bus 2 has three neighbours, bus 6 two
- **pipe networks, molecules, meshes** — the same shape of problem
- **keep the rule** — replace 'my eight neighbours' with 'whoever the edges connect'

TALK · 2 MIN Look at the six buses on the right. Which bus has the most neighbours, and why can no fixed 3×3 kernel be written for this picture?

THE SIX-BUS NETWORK



Bus 2 has three neighbours, bus 6 has two. No kernel fits that, but an update rule that sums over whatever neighbours exist does. This network returns in Ex_05 notebook 03 and in Ex_12.1.

Graph Matrices

NODE FEATURES AND ADJACENCY

- a **graph** — nodes and edges, written as two matrices
- **node features X** — one row per node, what was measured there
- **adjacency A** — a one wherever an edge exists; symmetric here
- **sparse** — 14 non-zeros of 36; real networks are far sparser

FIND IT · 1 MIN In the adjacency matrix on the right, which row belongs to bus 2, and which three columns of it are lit? Check against the six-bus picture.

ADJACENCY, FOR THE SIX BUSES

A

	1	2	3	4	5	6
1		1		1		
2	1		1		1	
3		1				1
4	1				1	
5		1		1		1
6			1		1	

Symmetric, because a line carries power both ways. The diagonal is empty because a bus is not connected to itself; graph convolution adds it back on purpose, two slides on.

Message Passing

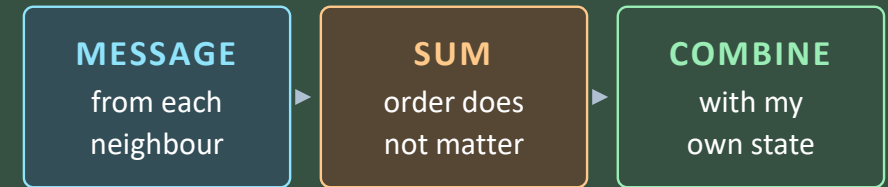
THREE STEPS, REPEATED AT EVERY NODE

- **for each node** — compute a message from each neighbour
- **sum the messages** — then combine with the node's own state
- **both functions are small networks** — shared across every node
- **the sum** — ignores order and count: one model, any graph size
- **K layers reach K hops** — match K to the graph diameter

$$\mathbf{h}_{i^{(l+1)}} = \phi \left(\mathbf{h}_{i^{(l)}}, \sum_{j \in \mathcal{N}(i)} \psi(\mathbf{h}_{i^{(l)}}, \mathbf{h}_{j^{(l)}}) \right)$$

$\mathbf{h}_i^{(l)}$ node i at layer l , $\mathcal{N}(i)$ its neighbours, ψ the message, ϕ the combine

ONE NODE'S UPDATE



Repeat for every node with the same two functions; repeat the layer to reach two hops.

HOW FAR INFORMATION TRAVELS

One layer reaches the neighbours, two layers their neighbours. For six buses the diameter is three, and notebook 03's sweep flattens at three layers. Beyond that the layers add parameters, not reach.

Graph Convolution

THE SIMPLEST CHOICE, IN ONE MATRIX LINE

- **the message** — the neighbour's state, unchanged
- **the combine step** — a shared linear map, then an activation
- **read right to left** — mix features, sum over neighbours, activate
- **add the identity** — so a node's own state is included
- **normalise by degree** — or sums grow with the neighbour count

$$\mathbf{H}^{(l+1)} = \sigma(\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{H}^{(l)} \mathbf{W}^{(l)})$$

TERM BY TERM

$\mathbf{H}$	node features, one row per node
$\mathbf{W}$	shared weights — the learned part
$\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$	neighbours, plus yourself
$\tilde{\mathbf{D}}$	degree — how many neighbours
$\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$	the normalised sum
σ	the activation

The same shape as a convolution: gather a neighbourhood, apply shared weights, activate. The adjacency matrix does the job the sliding window did.

Permutation Equivariance

RENUMBER THE NODES, THE OUTPUT FOLLOWS

- **numbering the buses is bookkeeping** — there is no natural order
- **a model whose answer changes when the nodes are renumbered** — is wrong, not merely inaccurate
- **a graph network** — permute the input and the output permutes, exactly
- **a dense network** — fails this, and notebook 03 measures by how much

$$f(\mathbf{PX}, \mathbf{PAP}^T) = \mathbf{P} f(\mathbf{X}, \mathbf{A})$$

$\mathbf{X}$ node features, $\mathbf{A}$ the adjacency, $\mathbf{P}$ any relabelling of the nodes, f the network

MEASURED IN EX_05 NOTEBOOK 03

graph network	1.7×10^{-16} rad
dense network	5.7×10^{-1} rad
after relabelling, GNN	accuracy unchanged
after relabelling, dense	153 × worse

The graph network's gap is floating-point noise: it is invariant, not approximately invariant. The dense network's answer is destroyed by an operation that changed nothing physical.

Choosing the Architecture

MEASURED IN NOTEBOOK 03

On the six-bus regression the dense baseline is more accurate than the graph network (0.0016 rad against 0.0043), and a DC linearisation is more accurate than both (0.0023).

WHY THE GRAPH NETWORK LOSES HERE

six nodes	a dense network can memorise the relationships
fixed topology	the structure is available exactly, not approximately
physics known	an admittance matrix already holds the topology

WHEN IT EARNS ITS PLACE

topology changes	the line-trip test degrades half as badly as dense
many networks	one model for hundreds of configurations
large graphs	cost grows with edges, not nodes squared
relabelling	operational data from several sources must not care

Name what the architecture buys in your formulation, not in general.

Challenges

ZERO MEANS TWO THINGS

missing measurements read as zero · add a mask channel beside the value · most of the difficulty in the state-estimation exercise

OVER-SMOOTHING

too many layers average every node to the same state · match depth to the graph diameter · residual connections (Li et al. 2018)

FIXED INPUT SIZE

a flatten-then-dense stack assumes one image size · global average pooling · resize at the input

DATA AND ENERGY EFFICIENCY

print	every tensor shape once
check	the adjacency is symmetric
permute	the nodes — does the output follow?
shrink	the depth to the diameter
mask	a channel for missing values
test	on a changed topology

The permutation test is four lines of code and tells you at once whether the model has the property it was built for. Kipf & Welling (2017); Li et al. (2018) on over-smoothing.

Exercise 5

NOTEBOOKS 01 TO 03, AND 04 AFTER L5.2

- **01** — a small CNN on generated inspection images: crack, pit, clean
- **02** — message passing by hand in NumPy, before any library
- **03** — node regression on the six buses, with the permutation test
- **04** — a sequence task, after the next lecture



THE QUESTION EACH NOTEBOOK ANSWERS

- 01** how many parameters does the CNN need against a dense network for the same accuracy?
- 02** what does the sum over neighbours do to the order and number of them?
- 03** does the output follow a permutation of the nodes, and by how much does dense fail?

REPORT QUESTION

The dense model was more accurate. Would you still use the graph network, and on what evidence?

Key Takeaways

1 What kind of network

Convolutional: feed-forward, with convolution layers. Graph: every node updated from its neighbours. Both: one local rule, everywhere.

2 What function it implements

Grid signal in, features or label out, shift-equivariant. Node features and adjacency in, an answer per node, edge or graph.

3 What the architecture is

Convolution and pooling blocks, then a dense head. K message-passing layers, then a readout, K near the graph diameter.

4 How it differs from a dense network

Local connections, shared weights, a parameter count independent of input size, a symmetry built in. Dense has none of these.

5 What it is used for

Images and signals: classify, segment, detect. Networks, meshes, molecules: node, edge, graph tasks. Dense wins on six fixed buses.

Next — L5.2: RNN, LSTM, attention and transformers.

Questions

TEN QUESTIONS FOR THE ORAL EXAMINATION

- 1 What kind of network is a convolutional network, and what function does it implement? *UDL ch. 10.1 – 10.2; GBC ch. 9.1*
- 2 Describe the architecture of a convolutional network, layer by layer: what happens to resolution and channels? *UDL ch. 10.3 – 10.4; Ex_05 notebook 01*
- 3 How does a convolutional network differ from a dense network in connections, weights, parameters and symmetry? *GBC ch. 9.2; Ex_05 notebook 01*
- 4 Given those differences, what are convolutional networks used for, and which task needs equivariance, not invariance? *UDL ch. 10.1, 10.5*
- 5 Why does a residual connection help the gradient, and what does a block with nothing to add learn? *UDL ch. 11.2; He et al. (2016)*
- 6 What kind of network is a graph network, and what function does it implement on node features and adjacency? *UDL ch. 13.1 – 13.3*
- 7 Describe the architecture of a graph network: the three steps of a layer, the readout, and K against the diameter? *UDL ch. 13.3, 13.5; Ex_05 notebook 02*
- 8 How does a graph network differ from a dense network, and what does permutation equivariance guarantee? *UDL ch. 13.2; Ex_05 notebook 03*
- 9 Given those differences, what are graph networks used for, and what is the difference between inductive and transductive? *UDL ch. 13.3, 13.6*
- 10 When is a graph network the wrong choice, and what did notebook 03 measure that shows it? *Ex_05 notebook 03*

NEXT

L5.2 — RNN, LSTM, Attention and Transformers

Today's structures were spatial, a grid and a graph. Next time the structure is temporal: how to carry information from the start of a long sequence to its end.

BEFORE THE NEXT SESSION

read	UDL ch. 12.1 – 12.3
run	Ex_05 notebooks 00 to 03
bring	your permutation-test numbers
think about	layer count against graph diameter

Bring the permutation numbers. L5.2 opens by comparing them with the room's.